

# 项目一报告

---

## Level1:配置YOLO环境

---

### 1.安装配置anaconda

\*搜索清华大学开源软件镜像站下载anaconda

\*添加环境变量

D:\Anaconda

D:\Anaconda\Scripts

\*验证

Win+R到cmd

conda--version

conda info

### 2.创建独立虚拟环境

\*从anaconda prompt里面写代码，创建是conda create -n 名称，激活是conda activate 名称、

\*遇到问题：一开始不懂环境的重要性，创建了四五个新环境，在不同环境里下载安装了不同东西

\*解决办法：从anaconda prompt里面写conda env list，回车，显示我所创建的所有环境，然后conda list 名称，即可看在不同环境里下载过的东西

### 3.判断无NVIDIA显卡，安装 CPU 版 PyTorch 及 YOLO 依赖

\*打开设备管理器查看是否有nvidia显卡，发现自身电脑是intel显卡，所以下载cpu版的pytorch

\*从官网搜索pytorch，下滑找到带有cpu only 的下载安装代码，将其在环境中粘贴安装

\*从anaconda prompt中写python，复制yolo task=detect mode=predict model=yolov8n.pt source=<https://ultralytics.com/images/zidane.jpg>代码，即配置成功YOLO v8环境

可以从文件夹中找到所用的图片，同时检验

## Level2:下载和标注数据集

---

### 1.下载CCPD数据集

### 2.转换标注为YOLO .txt格式

\*安装OpenCV

pip install opencv-python tqdm

pip install opencv-python matplotlib pillow -i <https://pypi.tuna.tsinghua.edu.cn/simple>

\*检验安装

pip install opencv-python matplotlib pillow -i <https://pypi.tuna.tsinghua.edu.cn/simple>

\*新建labels文件夹

\*打开anaconda prompt激活环境

\*新建记事本粘贴以下内容，然后打开pycharm

```
import os
import cv2
from tqdm import tqdm # 引入tqdm库用于显示进度条

def ccpd_file_parser(image_path):
    """
    解析CCPD数据集的单个文件名，提取边界框坐标。
    文件名格式示例: 025-95_113-154&383_386&473-386&473_154&383_95_113_363_323-0_0_2_27_28_33_31-73_168.jpg
    我们需要的是第三个 '-' 分割的部分: 154&383_386&473 (左下角和右上角坐标)
    """
    try:
        # 1. 按 '-' 分割文件名
        parts = image_path.stem.split('-')
```

```
        # 2. 获取边界框坐标部分
        bbox_part = parts[2]

        # 3. 按 '_' 分割，获取两个对角点坐标
        points = bbox_part.split('_')

        # 4. 解析第一个点（左下角）
        x1_str, y1_str = points[0].split('&')
        x1, y1 = int(x1_str), int(y1_str)

        # 5. 解析第二个点（右上角）
        x2_str, y2_str = points[1].split('&')
        x2, y2 = int(x2_str), int(y2_str)

        return [x1, y1, x2, y2]

    except (ValueError, IndexError) as e:
        print(f"文件名格式错误，无法解析: {image_path.name}. 错误: {e}")
        return None
```

```
def convert_to_yolo_format(bbox, img_width, img_height):
    """
```

```
    将[x1, y1, x2, y2]格式的坐标转换为YOLO格式。
    YOLO格式: <class_id> <x_center> <y_center>
    所有值都是相对于图像宽高的归一化值。
    """
```

```
    class_id = 0
```

```
    # 计算边界框的宽度和高度
```

```

box_w = bbox[2] - bbox[0]
box_h = bbox[3] - bbox[1]

# 计算边界框的中心点坐标
center_x = bbox[0] + box_w / 2
center_y = bbox[1] + box_h / 2

# 归一化
yolo_x = center_x / img_width
yolo_y = center_y / img_height
yolo_w = box_w / img_width
yolo_h = box_h / img_height

return f"{class_id} {yolo_x:.6f} {yolo_y:.6f} {yolo_w:.6f} {yolo_h:.6f}"

```

```
def process_ccpd_dataset(source_dir, output_dir):
```

```
    """
```

```
    处理整个CCPD数据集文件夹。
```

```
    """
```

```
    # 确保输出目录存在
```

```
    os.makedirs(output_dir, exist_ok=True)
```

```
    # 获取所有图片文件的路径
```

```
    image_files = list(Path(source_dir).glob("*.jpg"))
```

```
    if not image_files:
```

```
        print(f"警告: 在目录 '{source_dir}' 中未找到任何 .jpg 文件。")
```

```
        return
```

```
    print(f"找到 {len(image_files)} 张图片, 开始处理...")
```

```
    # 使用tqdm创建进度条
```

```
    for image_path in tqdm(image_files, desc="处理图片"):
```

```
        # 1. 读取图片获取尺寸
```

```
        try:
```

```
            img = cv2.imread(str(image_path))
```

```
            if img is None:
```

```
                print(f"\n警告: 无法读取图片 {image_path.name}, 跳过。")
```

```
                continue
```

```
            img_height, img_width, _ = img.shape
```

```
        except Exception as e:
```

```
            print(f"\n读取图片 {image_path.name} 时出错: {e}, 跳过。")
```

```
            continue
```

```
        # 2. 解析文件名获取边界框
```

```
        bbox = ccpd_file_parser(image_path)
```

```
        if bbox:
```

```
            # 3. 转换为YOLO格式
```

```
            yolo_string = convert_to_yolo_format(bbox, img_width, img_height)
```

```

        # 4. 构造标签文件名并保存
        label_filename = output_dir / (image_path.stem + ".txt")
        with open(label_filename, 'w') as f:
            f.write(yolo_string)

print("\n处理完成！")
print(f"所有标签文件已保存在：{output_dir}")

```

```

if name == 'main':
    from pathlib import Path

```

```

# --- 配置区域 ---
# 1. 设置您的CCPD图片所在的源文件夹
# 请确保使用正斜杠 '/' 或双反斜杠 '\\'
SOURCE_IMAGES_DIR = "D:/CCPD_Dataset/CCPD2019"

# 2. 设置您希望保存.txt标签文件的目标文件夹
OUTPUT_LABELS_DIR = "D:/CCPD_Dataset/labels"
# --- 配置结束 ---

# 检查tqdm是否已安装
try:
    from tqdm import tqdm
except ImportError:
    print("警告：tqdm 库未安装。进度条将不可用。")
    print("您可以通过 'pip install tqdm' 来安装它。")
    # 创建一个虚拟的tqdm函数，以便脚本在没有tqdm的情况下也能运行
    def tqdm(iterable, *args, **kwargs):
        return iterable

# 运行主处理函数
process_ccpd_dataset(Path(SOURCE_IMAGES_DIR), Path(OUTPUT_LABELS_DIR))``

```

## Level3:划分数数据集，配置并训练模型

### 1.划分数数据集

\*使用记事本输入代码后保存，打开pycharm打开记事本

```

import os
import shutil
import random
from pathlib import Path
from tqdm import tqdm

```

```

def split_dataset(source_img_dir, source_lbl_dir, output_dir, split_ratio=0.8):
    """
    将源数据集随机划分为训练集和验证集，并移动到新的目录结构中。

    参数：
    """

```

```

- source_img_dir (Path): 存放所有源图片的文件夹路径。
- source_lbl_dir (Path): 存放所有源标签的文件夹路径。
- output_dir (Path): 用于存放划分后数据集的新文件夹的根路径。
- split_ratio (float): 训练集所占的比例。
"""

# 1. 创建输出目录结构
train_images_path = output_dir / "images" / "train"
val_images_path = output_dir / "images" / "val"
train_labels_path = output_dir / "labels" / "train"
val_labels_path = output_dir / "labels" / "val"

print("正在创建输出目录...")
for path in [train_images_path, val_images_path, train_labels_path, val_labels_path]:
    path.mkdir(parents=True, exist_ok=True)
print("目录创建完成! ")

# 2. 获取所有图片文件列表
image_files = sorted([p for p in source_img_dir.glob("*.jpg")])

if not image_files:
    print(f"警告: 在目录 '{source_img_dir}' 中未找到任何 .jpg 文件。脚本将退出。")
    return

# 3. 随机打乱文件列表
random.seed(42) # 设置随机种子以保证每次划分结果一致
random.shuffle(image_files)

# 4. 计算划分点
split_point = int(len(image_files) * split_ratio)
train_files = image_files[:split_point]
val_files = image_files[split_point:]

print(f"总共找到 {len(image_files)} 张图片。")
print(f"划分为: {len(train_files)} 张训练图片, {len(val_files)} 张验证图片。")

# 5. 移动文件
print("\n正在移动训练集文件...")
for image_path in tqdm(train_files, desc="移动训练集"):
    label_path = source_lbl_dir / (image_path.stem + ".txt")
    if label_path.exists():
        shutil.move(str(image_path), str(train_images_path))
        shutil.move(str(label_path), str(train_labels_path))
    else:
        print(f"警告: 找不到图片 '{image_path.name}' 对应的标签文件, 已跳过。")

print("\n正在移动验证集文件...")
for image_path in tqdm(val_files, desc="移动验证集"):
    label_path = source_lbl_dir / (image_path.stem + ".txt")
    if label_path.exists():
        shutil.move(str(image_path), str(val_images_path))
        shutil.move(str(label_path), str(val_labels_path))
    else:

```

```

        print(f"警告：找不到图片 '{image_path.name}' 对应的标签文件，已跳过。")

    print("\n数据集划分完成！")
    print(f"新的数据集已保存在：{output_dir}")

if __name__ == '__main__':
    # --- 配置区域 ---
    # 请根据您的实际情况修改以下路径

    # 1. 包含所有原始图片的文件夹（ccpd_parser.py脚本的SOURCE_IMAGES_DIR）
    SOURCE_IMAGES_DIR = Path(r"D:\yolo\yolo\train\dataset\mydata\images\train")

    # 2. 包含所有原始标签的文件夹（ccpd_parser.py脚本的OUTPUT_LABELS_DIR）
    SOURCE_LABELS_DIR = Path(r"D:\yolo\yolo\train\dataset\mydata\labels\train")

    # 3. 您希望创建并存放最终划分好的数据集的新文件夹
    # 建议使用 'dataset' 或 'License_Plate_Dataset' 等名称
    OUTPUT_DATASET_DIR = Path(r"D:\yolo\yolo\train\dataset\mydata\dataset")

    # 4. 训练集所占的比例
    TRAIN_RATIO = 0.8
    # --- 配置结束 ---

    # 运行主函数
    split_dataset(SOURCE_IMAGES_DIR, SOURCE_LABELS_DIR, OUTPUT_DATASET_DIR, TRAIN_RATIO)

```

2.创建mydata.yaml配置文件

3.训练模型

\*(chuangjian) D:\yolo\yolo\train\dataset\mydata\dataset>yolo train data=mydata.yaml model=yolov8n.pt epochs=100 imgsz=640 batch=64

## Level4：模型推理

1.对本地图片进行推理

\*在pycharm中新建并输入

```

from ultralytics import YOLO
import cv2

model = YOLO(r'D:\yolo\yolo\train\dataset\mydata\dataset\runs\detect\train10\weights\best.pt')

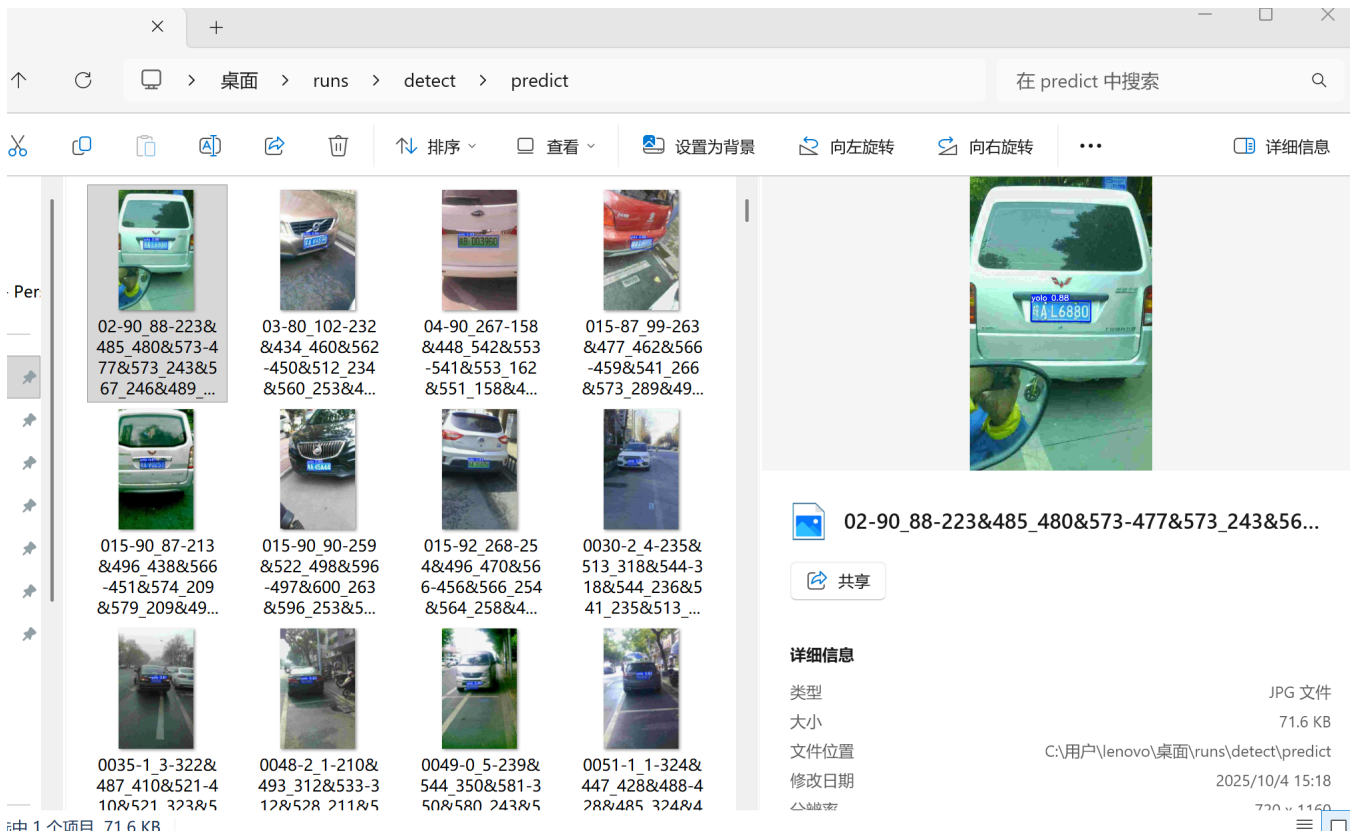
image_path = r'D:\yolo\yolo\test\images'

results = model.predict(source=image_path, save=True)

print("推理完成！ 请检查 runs/detect/predict 文件夹下的结果。")

*然后运行得到新文件夹predict

```



## 2.对视频进行推理

\*继续在pycharm里输入

```
from ultralytics import YOLO
```

```
import cv2
```

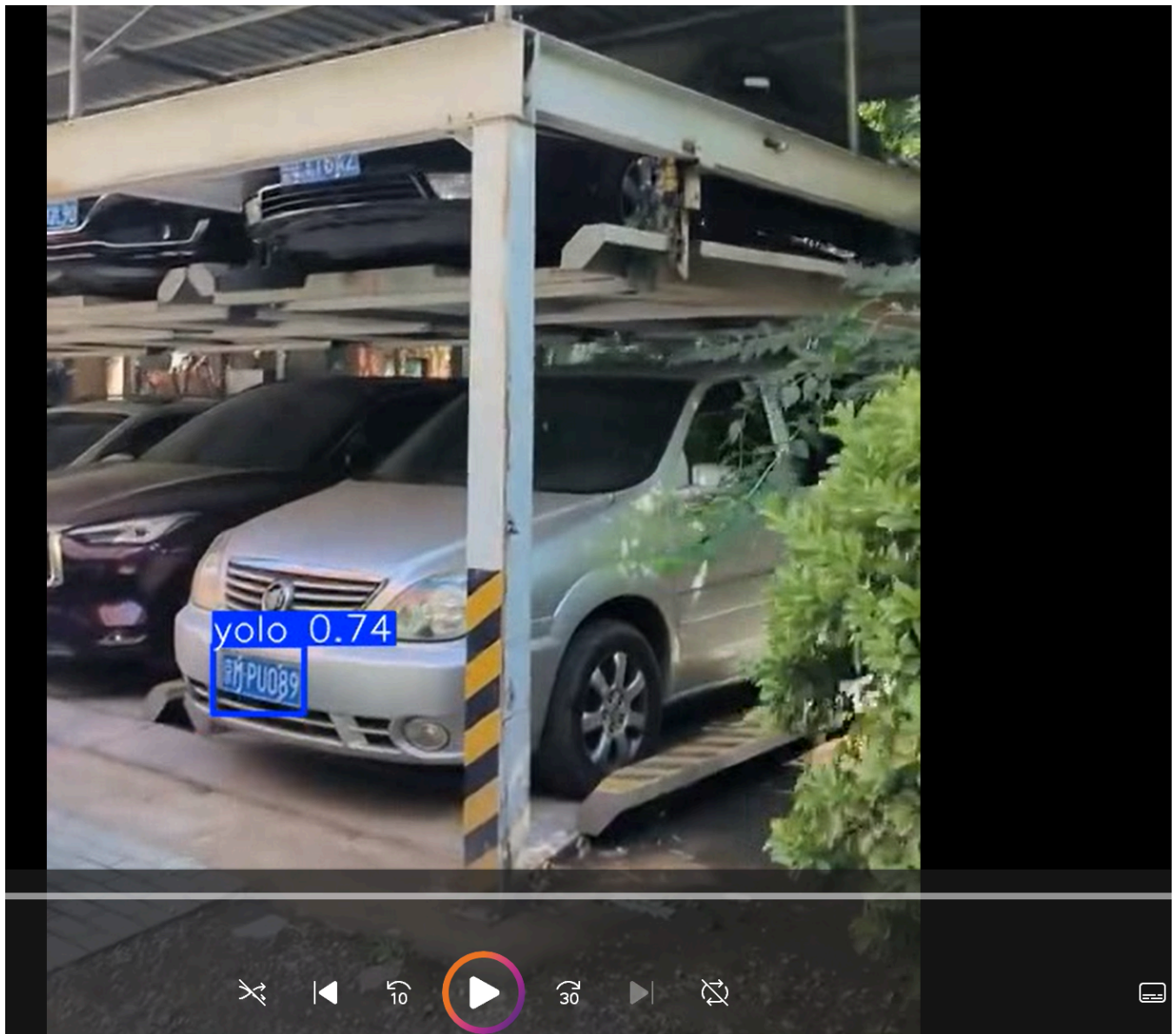
```
model = YOLO(r'D:\yolo\yolo\train\dataset\mydata\dataset\runs\detect\train10\weights\best.pt')
```

```
video_path = r'D:\yolo\yolo\test\video'
```

```
results = model.predict(source=video_path, save=True)
```

```
print("视频推理完成！请检查 runs/detect/predict 文件夹下的结果。")
```

\*然后运行



### 3.对摄像头进行推理

\*在pycharm里继续输入

```
from ultralytics import YOLO
import cv2
```

```
model = YOLO(r'D:\yolo\yolo\train\dataset\mydata\dataset\runs\detect\train10\weights\best.pt')
```

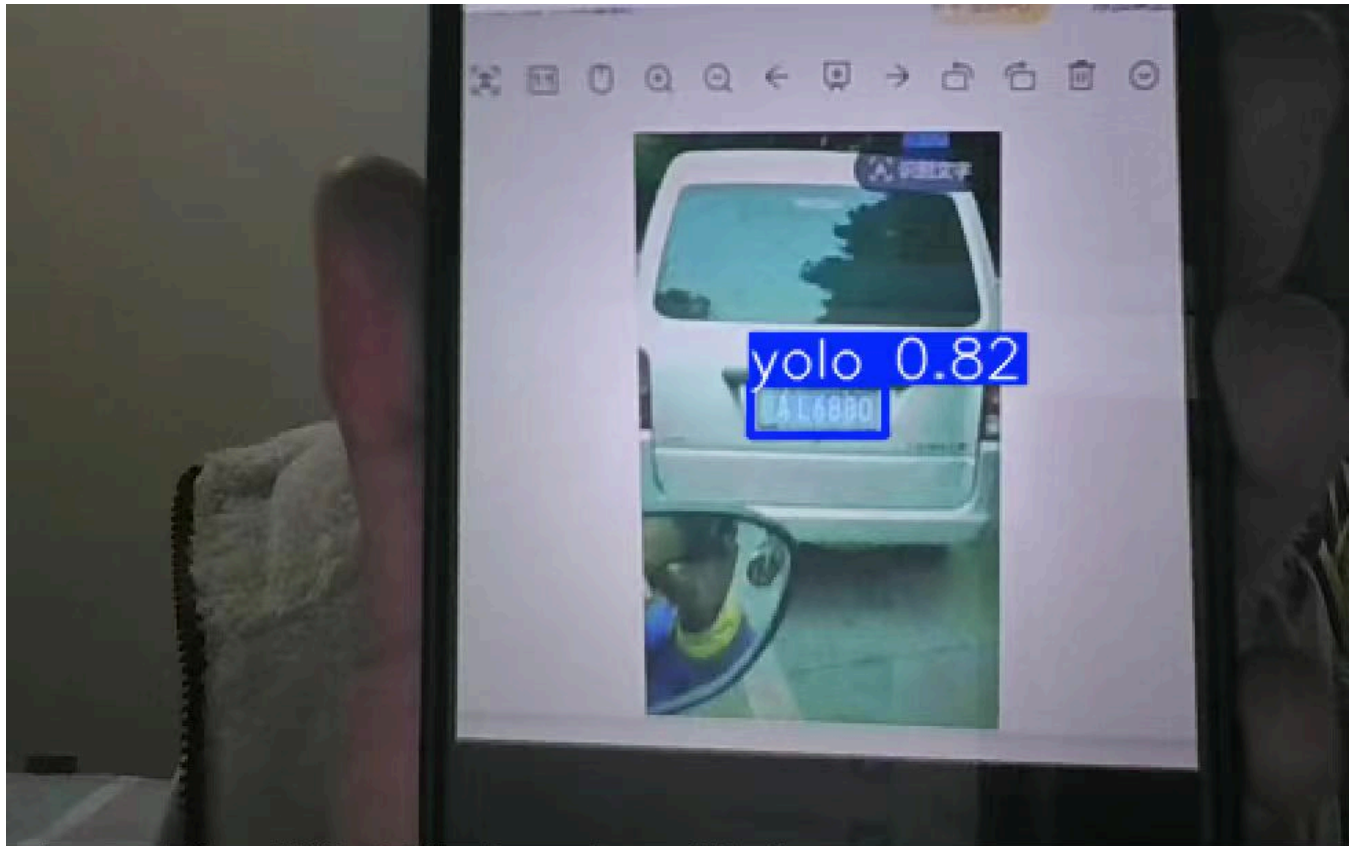
```
results = model.predict(source="0", show=True, stream=True)
```

```
for r in results:
```

```
    pass
```

\*运行得到弹出的摄像头视频





> 3,446

|    |         |   |       |        |
|----|---------|---|-------|--------|
| 0: | 480x640 | 1 | yolo, | 39.8ms |
| 0: | 480x640 | 1 | yolo, | 40.7ms |
| 0: | 480x640 | 1 | yolo, | 44.1ms |
| 0: | 480x640 | 1 | yolo, | 44.5ms |
| 0: | 480x640 | 1 | yolo, | 41.7ms |
| 0: | 480x640 | 1 | yolo, | 41.8ms |
| 0: | 480x640 | 1 | yolo, | 40.0ms |
| 0: | 480x640 | 1 | yolo, | 41.4ms |
| 0: | 480x640 | 1 | yolo, | 49.9ms |
| 0: | 480x640 | 1 | yolo, | 53.5ms |
| 0: | 480x640 | 1 | yolo, | 46.5ms |
| 0: | 480x640 | 1 | yolo, | 45.8ms |